

Script de Soutenance

Microservice G6 — Paiement | SGITU

#	Etape	Action rapide	Fait ?
01	Présentation du microservice G6	Slide 1 (titre) + Slide 3 (contexte)	■
02	Stack technique et organisation en couches	Slide Architecture Technique	■
03	Ajout d'une carte bancaire et génération OTP	Swagger → POST /payment-accounts/card Logs Docker	■
04	Vérification OTP → statut ACTIVE	Swagger → POST /payment-accounts/{id}/verify-otp	■
05	Créer un paiement et consulter la facture	Swagger → POST /payments GET /payments/43/invoice	■
06	Communication Kafka + REST callbacks	Logs Docker + Slide Intégration + phpMyAdmin table payments	■
07	JWT, HTTPS/TLS et Grafana	Swagger → GET /payments/user/1 sans token → 401 http://localhost:3000	■

01 — Présentation du microservice G6

INTRODUCTION

Action : Slide 1 (titre) + Slide 3 (contexte)

"Notre microservice G6 est responsable du cycle de paiement complet dans le système SGITU. Il gère les transactions, les moyens de paiement carte et Mobile Money, la vérification OTP, la génération automatique de factures, et les remboursements. Le service expose une API REST sécurisée par JWT et communique avec les autres microservices via Kafka et REST callbacks."

Conseil : Pointez le diagramme de contexte sur la slide.

02 — Stack technique et organisation en couches

ARCHITECTURE

Action : Slide Architecture Technique

"Notre service est développé en Java 17 avec Spring Boot 3, organisé en couches : Controllers REST, Services Métier, et Repositories Spring Data JPA. Pour la sécurité on utilise JWT avec HTTPS/TLS. Pour la communication asynchrone, Apache Kafka. Et pour l'observabilité, Prometheus et Grafana avec Spring Actuator."

Conseil : Pointez chaque couche sur le schéma.

03 — Ajout d'une carte bancaire et génération OTP

DEMO — CARTE +
OTP

Action : Swagger → POST /payment-accounts/card | Logs Docker

"J'ajoute une carte bancaire VISA pour l'utilisateur 5. Le service crée un compte de paiement avec le statut PENDING_VERIFICATION. Automatiquement, le système génère un OTP et publie un événement Kafka sur le topic payment.notification avec le type PAYMENT_METHOD_OTP — on le voit ici dans les logs. C'est G5 Notifications qui consomme cet événement pour envoyer l'email à l'utilisateur."

Conseil : Body : cardNumber 4532015112830366, userId 5, email test@gmail.com

04 — Vérification OTP → statut ACTIVE

DEMO —
VERIFICATION OTP

Action : *Swagger → POST /payment-accounts/{id}/verify-otp*

"Je vérifie maintenant l'OTP reçu. Le statut du compte passe de PENDING_VERIFICATION à ACTIVE. La carte VISA se terminant par 0366, avec un solde de 1000 MAD, est maintenant prête pour les transactions. C'est une mesure de sécurité importante — aucun paiement n'est possible sans cette validation préalable."

Conseil : OTP visible dans les logs Docker : docker-compose logs payment-service

05 — Créer un paiement et consulter la facture

DEMO — PAIEMENT +
FACTURE

Action : *Swagger → POST /payments | GET /payments/43/invoice*

"Je crée un paiement de 50 MAD pour un ticket. Le service vérifie le solde de la carte, débite le montant, et retourne le statut SUCCESS avec un paymentId et un invoiceId. Voici la facture INV-PAY-43 générée automatiquement — elle contient le numéro de facture, le montant, la méthode de paiement CARD, et la date d'émission. Dans les logs on voit que deux événements Kafka ont été publiés : INVOICE_GENERATED et PAYMENT_SUCCESS."

Conseil : savedPaymentToken : CARD-TOKEN-C75D2F90 | sourceType : TICKET

06 — Communication Kafka + REST callbacks

INTEGRATION
INTER-SERVICES

Action : *Logs Docker + Slide Intégration + phpMyAdmin table payments*

"Notre service communique avec trois microservices. Avec G5 Notifications via Kafka asynchrone — les événements sont publiés avec acks=all et retries=3 pour garantir la livraison. Avec G1 Billetterie et G2 Abonnements via REST callbacks après chaque paiement. Dans la base de données on voit des entrées G1-TICKET, G1-CANCEL, G1-REFUND — preuve que ces intégrations ont fonctionné avec succès."

Conseil : Montrez la table payments dans phpMyAdmin pendant l'explication.

Action : Swagger → GET /payments/user/1 sans token → 401 | http://localhost:3000

"Pour la sécurité, toutes les requêtes passent par notre JwtAuthenticationFilter. Sans token on obtient un 401 Unauthorized — je le montre ici. Avec token valide, l'accès est autorisé. Le service tourne en HTTPS avec un certificat TLS PKCS12. Pour l'observabilité, Prometheus scrape nos métriques Spring Actuator, et Grafana affiche en temps réel : requêtes, latence, erreurs, et métriques JVM."

Conseil : Ouvrez Grafana sur http://localhost:3000 pour montrer les dashboards.

Checklist finale

✓ JWT fonctionnel — 401 sans token, 200 avec token	
✓ Carte ajoutée — POST /payment-accounts/card → 201	
✓ OTP vérifié — statut ACTIVE	
✓ Paiement SUCCESS — POST /payments → 201	
✓ Facture générée — INV-PAY-43	
✓ Kafka — PAYMENT_METHOD_OTP + INVOICE_GENERATED + PAYMENT_SUCCESS	
✓ Base de données — G1-TICKET, G1-REFUND visibles	
✓ Grafana — dashboards temps réel sur localhost:3000	